

Curso Inicial Front-End

Clase 15: Arrays y Métodos de Iteración

Módulo 2: Dominio de JavaScript y Lógica de Programación



Presentación

En el mundo real, los datos nunca vienen solos. En esta clase aprenderemos a trabajar con **Arrays (Arreglos)**, la estructura que permite agrupar y manipular colecciones de datos. Dominaremos el "Catálogo Maestro" de métodos modernos como `.map()`, `.filter()` y `.reduce()`, que te permitirán procesar miles de datos con apenas unas líneas de código.



Objetivos

- Dominar la manipulación de colecciones de datos mediante el índice y la longitud.
- Implementar métodos de iteración modernos para transformar, filtrar y buscar datos.
- Comprender y aplicar el **Encadenamiento (Chaining)** para resolver problemas complejos de forma elegante.



Bloques Temáticos

- Tema 1: Operaciones CRUD en arreglos.
 - El Concepto de Índice: El Tren de Pasajeros
 - Operaciones con el Tren (Métodos Mutables Clásicos)
 - 3. Métodos Básicos de Arrays (Modificando la Fila del Supermercado)
- Tema 2: Iteración moderna: for...of y métodos básicos (.push, .pop, .shift, .unshift).
 - Los Métodos Modernos de Iteración (Los Inmutables)

Imagina que eres un profesor y tienes 50 alumnos nuevos en tu clase. Si quisieras guardar sus nombres en la memoria del programa para poder trabajar con ellos, tendrías que declarar `let alumno1 = "Juan"`, luego `let alumno2 = "Ana"`, ¡cincuenta veces! Eso es insostenible.

Para agrupar colecciones de datos, los lenguajes de programación crearon una Estructura de Datos especial: el **Array** (o Arreglo). Técnicamente, un Array es una lista ordenada de información bajo un solo nombre de variable.

 **Recurso Oficial:** [MDN Web Docs - Arrays en JavaScript](#)

Tema 1: Operaciones CRUD en arreglos.

1. El Concepto de Índice: El Tren de Pasajeros

Imagina que un Array es físicamente un tren de carga con muchos vagones alineados. Cada vagón puede guardar un dato distinto (un número, un texto o incluso un objeto entero).

¿Por qué los programadores empiezan a contar desde cero?

Es la duda número 1 de todo estudiante que ingresa al mundo de la informática. Para el "cerebro" de la computadora (la Memoria RAM), el identificador no es el "número de vagón", sino la **distancia** matemática que hay desde el inicio del tren.

Esa distancia se llama **Índice (Index)**.

- El primer vagón está pegado a la locomotora, a **0 metros** de distancia. Por eso su índice es **0**.
- El segundo vagón está a **1 metro** de distancia. Por eso su índice es **1**.

En lenguaje simple: Un Array guarda los datos uno al lado del otro en la memoria. Como la computadora sabe dónde empieza la lista, puede encontrar cualquier elemento al instante sin importar si la lista es corta o gigante ($O(1)$).

Traducción Técnica: Un Array es una estructura de datos indexada que reside en bloques contiguos de memoria. El índice `0` representa el desplazamiento o **offset** nulo desde el puntero base del array. El acceso mediante índice tiene una complejidad temporal de **O(1)**, lo que garantiza una velocidad constante independientemente del tamaño del array. En **JavaScript**, los arrays son objetos dinámicos que pueden almacenar tipos mixtos y redimensionarse automáticamente.

```
/* Observa los corchetes [ ]. Son el símbolo universal de que estás creando un Array. */
const frutas = ["🍎", "🍌", "🍋"];
// Distancias:      0      1      2

/* Usamos los corchetes [] pegados al nombre de la variable para acceder al vagón. */
console.log(frutas[0]); // El motor abre el vagón número 0 e imprime:
"🍎"

/* La propiedad mágica '.length' (Longitud) nos dice cuántos vagones hay en total. */
console.log(frutas.length); // Imprimirá: 3 (Porque hay 3 vagones físicos).
```

2. Operaciones con el Tren (Métodos Mutables Clásicos)

JavaScript nos regala "Métodos" (funciones ya construidas por dentro del sistema) para enganchar o desenganchar vagones rápidamente.

- **.push(dato)**: Engancha un vagón al final absoluto de la fila. (El más usado).
- **.pop()**: Desengancha de golpe el último vagón y lo tira a la basura.
- **.unshift(dato)**: Empuja todo hacia atrás y engancha un vagón al principio (índice 0).
- **.shift()**: Corta y elimina violentamente el primer vagón de la fila.

Mutabilidad vs Inmutabilidad (Concepto vital para el futuro en React)

En la programación moderna, alterar (mutar) la información original es considerado un pecado capital. - **Método Mutable (Destructivo)**: Rompe la variable original y la altera para siempre. Todos los métodos mencionados arriba (como `.push()`) destruyen la historia original de tu tren. En el futuro, si usas estos métodos en React.js, la pantalla de tu usuario se va a congelar porque React no detecta cambios violentos por la espalda.

- **Método Inmutable (Incorruptible)**: Le toma una foto a tu tren original, la copia, le aplica los cambios solicitados y **te devuelve un tren 100% nuevo y clonado**, dejando tu variable original pura e intacta.

3. Métodos Básicos de Arrays (Modificando la Fila del Supermercado)

Imagina que un **Array** es como la fila de la caja de un **supermercado**. Las personas están ordenadas y cada una tiene una posición (índice) **empezando desde el 0**. JavaScript nos da herramientas nativas para **agregar** o **quitar elementos** de esta **fila** de forma muy sencilla.

Agregar y Quitar al Final (Los más rápidos)

- **.push(elemento):** Agrega uno o más elementos al final de la fila.
- **.pop():** Quita el último elemento de la fila y te lo devuelve. Como ya se va del supermercado, la fila se acorta.

```
Establecemos nuestro array base:
let fila = ["Hugo", "Paco"];

// Agregar al final un nuevo elemento:
fila.push("Luis");
console.log(fila); // ["Hugo", "Paco", "Luis"]

// Quitar el último elemento:
let clienteAtendido = fila.pop();
console.log(clienteAtendido); // "Luis" (el que salió de la fila)
console.log(fila);           // ["Hugo", "Paco"]
```

Agregar y Quitar al Principio (Los más costosos)

*Agregar o quitar al inicio obliga a que todos los demás elementos de la fila tengan que **"correrse"** un lugar, **reconfigurando sus índices**.*

- **.unshift(elemento):** Inserta un elemento al principio de la fila (índice 0).
- **.shift():** Remueve el primer elemento de la fila y te lo entrega.

```
Establecemos nuestro array base:
let fila = ["Hugo", "Paco"];

// Agregar elemento al inicio del array:
fila.unshift("Luis");
console.log(fila); // ["Luis", "Hugo", "Paco"]
```

```
// Quitar el primer elemento del array:
let coladoAtendido = fila.shift();
console.log(coladoAtendido); // "Luis" (el que estaba primero)
console.log(fila);           // ["Hugo", "Paco"]
```

Iteración Moderna con for...of (El Escáner Automático)

Cuando queremos revisar uno por uno los elementos de un array, la forma antigua de escribir un bucle for requería configurar **contadores** ($i = 0$), **condiciones de parada** ($i < \text{array.length}$) e **incrementos** ($i++$). Es fácil equivocarse en la lógica matemática y romper el programa.

El estándar moderno nos trae **for...of**, que funciona como un escáner que recorre el array automáticamente de principio a fin, **entregándote** el elemento listo para usar en cada vuelta:

```
const compras = ["Manzanas", "Leche", "Café"];

// ❌ Forma antigua (Propensa a errores de índice):
// for (let i = 0; i < compras.length; i++) {
//   console.log(compras[i]);
// }

// ✅ Forma moderna con for...of (Simple y directa):
for (const producto of compras) {
  console.log(`Tengo que comprar: ${producto}`);
}
```


- *¿Cómo leerlo en español?: "**Para cada producto que esté dentro de compras, hacé lo siguiente...**"*
- *Ventaja: No necesitas manejar **números, índices**, ni preocuparte por cuándo **termina** el array; JavaScript se **encarga de todo**.*

Tema 2: Iteración moderna: for...of y métodos básicos (.push, .pop, .shift, .unshift).

1. Los Métodos Modernos de Iteración (Los Inmutables)

Llegamos al estándar de la industria. Estos son métodos Inmutables que recorren (iteran) vagón por vagón de forma automática, sin que nosotros tengamos que hacer matemáticas complejas.

.map() – La Fábrica Transformadora

Es el rey absoluto de **React**. Es una máquina que recibe cada vagón individualmente, le hace un cambio, y lo inyecta en un tren paralelo clonado.

Mapeo de Sintaxis (Desmitificando la lectura):

```
const preciosOriginales = [100, 200, 300];

/*
1. '.map()' arranca un bucle interno automático.

2. '(precioActual)' es un parámetro inventado por ti que representa al
vagón individual que está siendo procesado en ese milisegundo.

3. La flecha '=>' apunta al cálculo que quieres hacer (multiplicarlo por
2).
```

```
*/  
const preciosConInflacion = preciosOriginales.map((precioActual) =>  
  precioActual * 2);  
  
console.log(preciosConInflacion); // Array Clonado y modificado: [200,  
  400, 600]  
console.log(preciosOriginales); // Array Original (Sigue intacto, ¡es  
  immutable!): [100, 200, 300]
```

.filter() – El Colador de Calidad de Datos

Recorre todo el tren evaluando una condición matemática. Si el vagón da **true**, lo copia al nuevo tren. Si da **false**, lo rechaza. Es ideal para crear buscadores.

```
const edadesEstudiantes = [15, 20, 12, 30, 17];  
/* 'edad' representa a cada alumno individual iterado. La condición lógica  
  debe ir después de la flecha => */  
const alumnosMayores = edadesEstudiantes.filter(edad => edad >= 18);  
// Resultado en 'alumnosMayores': [20, 30]
```

Checkpoint de Comprensión (La trampa del **.sort()):** Tienes un Array numérico desordenado: `let puntajes = [10, 2, 30]`; y le aplicas el viejo método destructivo **.sort()** intentando ordenarlos de menor a mayor. Sin embargo, al imprimirlo, obtienes este desastre: `[10, 2, 30]`. Técnicamente, **.sort()** convierte todo el Array a texto **String** antes de comparar. Sabiendo esto, lógicamente, ¿por qué crees que la computadora pensó que el texto **"10"** era menor que el texto **"2"**? (Pista: Piensa en cómo el abecedario ordena alfabéticamente la letra A y la letra B, comparado con el número 1 y el número 2).

Catálogo Maestro de Métodos de Array

En el desarrollo moderno, evitamos usar bucles **for** manuales. Usamos métodos que hacen el trabajo por nosotros:

Método	Función	Ejemplo de Código
<code>.push()</code>	Agrega al final.	<code>arr.push("Uva"); // ["Manzana", "Uva"]</code>
<code>.pop()</code>	Quita el último.	<code>arr.pop(); // ["Manzana"]</code>
<code>.map()</code>	Transforma cada uno.	<code>precios.map(p => p * 1.2); // [12, 24]</code>
<code>.filter()</code>	Filtra por condición.	<code>edades.filter(e => e >= 18); // [20, 25]</code>
<code>.find()</code>	Busca el primero.	<code>users.find(u => u.id === 1); // {id:1, nom:"Ana"}</code>
<code>.reduce()</code>	Acumula en uno solo.	<code>num.reduce((a, b) => a + b, 0); // 100</code>
<code>.forEach()</code>	Ejecuta para cada uno.	<code>arr.forEach(x => console.log(x));</code>
<code>.some()</code>	¿Al menos uno cumple?	<code>arr.some(p => p.precio < 10); // true</code>
<code>.every()</code>	¿Todos cumplen?	<code>arr.every(p => p.stock > 0); // false</code>

Ejemplo Pro: Encadenamiento (Chaining)  ¡El poder máximo de **JavaScript**! Como los métodos inmutables siempre te devuelven un tren totalmente nuevo, puedes pegarlos uno detrás del otro usando un punto.

En lenguaje simple: El encadenamiento es como una línea de ensamblaje: el primer método limpia el dato, se lo pasa al segundo que lo transforma, y el tercero lo entrega terminado. No tienes que crear variables intermedias para cada paso.

Traducción Técnica: El **Chaining** es un patrón de diseño que permite invocar múltiples métodos en una sola instrucción. Esto es posible porque cada método inmutable (como `.filter` o `.map`) devuelve una nueva instancia de la misma clase (un nuevo Array), permitiendo que el siguiente método se ejecute sobre el resultado del anterior.

```
const totalVentas = productos
  .filter(p => p.precio > 100) // 1. Me quedo con los caros
  .map(p => p.precio)          // 2. Extraigo solo los números
  .reduce((a, b) => a + b, 0); // 3. Sumo el total
```

Documentación Oficial (Referencias)

- **JS:** <https://developer.mozilla.org/es/docs/Web/JavaScript>
- **JavaScript:** <https://developer.mozilla.org/es/docs/Web/JavaScript>
- **React:** <https://es.react.dev/>